🚀 Node.js - Preguntas Frecuentes en Entrevistas

Este documento contiene una recopilación de preguntas frecuentes y avanzadas sobre **Node.js** que suelen aparecer en entrevistas para desarrolladores, con respuestas detalladas y ejemplos prácticos.

🔹 **Cada pregunta está explicada en detalle en este mismo archivo.**
Puedes recorrerlo fácilmente usando el índice. 🚀

📌 Índice

1. ¿Cómo funciona el Event Loop en Node.js?
2. ¿Cuál es la diferencia entre `process.nextTick()` y `setImmediate()`?
3. ¿Qué es el clustering en Node.js y cuándo usarlo?
4. ¿Cómo gestionar memoria y detectar memory leaks?
5. ¿Qué es el event-driven non-blocking I/O?
6. ¿Qué diferencias hay entre streams y buffers?
7. ¿Cómo manejar errores correctamente (callback, promesas y `async/await`)?
8. ¿Qué son middlewares y cómo funcionan en frameworks como Express?
9. ¿Qué es el patrón de diseño Factory y cómo se usa en Node.js?
10. ¿Qué es una arquitectura de microservicios y cómo aplicarla en Node.js?
11. Estrategias para manejar autenticación y autorización
12. ¿Cómo optimizar el rendimiento de una API Node.js?
13. ¿Qué es la política Same-Origin y CORS?
14. ¿Cómo implementar caching efectivo en Node.js?
15. ¿Cómo hacer debugging avanzado en Node.js?
16. ¿Qué es el Garbage Collector en Node.js y cómo monitorearlo?
17. ¿Qué herramientas existen para profiling y performance testing?

📌 Preguntas y Respuestas

🔹 ¿Cómo funciona el Event Loop en Node.js?

El Event Loop permite a Node.js manejar múltiples operaciones asincrónicas en un solo hilo. Está compuesto por varias fases (timers, I/O callbacks, idle, poll, check y close callbacks).

Cuando una operación asincrónica (como leer un archivo o una solicitud HTTP) finaliza, su callback se coloca en una cola correspondiente y el Event Loop la ejecuta cuando llega la fase adecuada.

```javascript
```

```
setTimeout(() => console.log("Timer"), 0);
setImmediate(() => console.log("Immediate"));
process.nextTick(() => console.log("Next Tick"));
```

Salida típica:

```
Next Tick
Timer
Immediate
```

🔹 ¿Cuál es la diferencia entre process.nextTick() y setImmediate()?

- `process.nextTick()`: Ejecuta su callback **antes de que el Event Loop pase a la siguiente fase**.
- `setImmediate()`: Ejecuta su callback **durante la fase 'check'** del Event Loop.

`process.nextTick()` siempre tiene prioridad sobre `setImmediate()`.

🔹 ¿Qué es el clustering en Node.js y cuándo usarlo?

El clustering permite crear múltiples procesos (workers) que comparten el mismo puerto, permitiendo a Node.js usar múltiples núcleos de CPU.

```javascript
const cluster = require('cluster');
const http = require('http');
const os = require('os');

if (cluster.isMaster) {
  os.cpus().forEach(() => cluster.fork());
} else {
  http.createServer((req, res) => res.end('Hello World')).listen(3000);
}
```

Se usa en aplicaciones de alto tráfico para mejorar la escalabilidad y tolerancia a fallos.

🔹 ¿Cómo gestionar memoria y detectar memory leaks?

- Usar `process.memoryUsage()` para monitorear consumo.
- Herramientas como Chrome DevTools, `heapdump` y `clinic.js`.
- Detectar patrones sospechosos: acumulación de listeners, referencias circulares o uso excesivo de variables globales.

🔹 ¿Qué es el event-driven non-blocking I/O?

Es un modelo donde las operaciones de entrada/salida (I/O) no bloquean el hilo principal. Node.js usa callbacks, Promesas y el Event Loop para manejar operaciones asincrónicas eficientemente.

🔹 ¿Qué diferencias hay entre streams y buffers?

Concepto	Streams	Buffers
Propósito	Manejar datos en movimiento	Almacenar datos binarios
Ejemplo	Lectura de archivos grandes	Contenido de archivos pequeños

Streams permiten procesar datos por fragmentos, evitando cargar todo en memoria.

🔹 ¿Cómo manejar errores correctamente (callback, promesas y async/await)?

- **Callback**: Primer argumento `err`.
- **Promesas**: `.catch()` para manejar errores.
- **Async/Await**: Bloques `try...catch`.

```javascript
async function fetchData() {
  try {
    const result = await someAsyncCall();
  } catch (error) {
    console.error('Error:', error);
  }
}
```

🔹 ¿Qué son middlewares y cómo funcionan en frameworks como Express?

Son funciones que tienen acceso al objeto `request`, `response` y a la función `next()`. Permiten manejar peticiones, respuestas, autenticación y manejo de errores.

```javascript
app.use((req, res, next) => {
  console.log('Middleware ejecutado');
  next();
});
```

🔹 ¿Qué es el patrón de diseño Factory y cómo se usa en Node.js?

Es un patrón que usa una función para crear objetos en lugar de usar `new`

directamente.

```javascript
function createUser(name) {
  return { name, role: 'user' };
}
```

Se usa para crear objetos dinámicamente con lógica personalizada.

🔹 ¿Qué es una arquitectura de microservicios y cómo aplicarla en Node.js?

Divide una aplicación en servicios pequeños e independientes que se comunican mediante HTTP, mensajes o eventos.

En Node.js, se puede implementar con frameworks como Express, Fastify o Hapi y herramientas como RabbitMQ o Kafka para la mensajería.

🔹 Estrategias para manejar autenticación y autorización

- **Autenticación**: JWT, OAuth2, Passport.js.
- **Autorización**: Roles y permisos a nivel de middleware o controlador.
- Usar HTTPS, limitar intentos de inicio de sesión y validar tokens correctamente.

🔹 ¿Cómo optimizar el rendimiento de una API Node.js?

- Usar clustering y balanceo de carga.
- Implementar caching.
- Optimizar consultas a base de datos.
- Limitar el tamaño de las respuestas.

🔹 ¿Qué es la política Same-Origin y CORS?

La política Same-Origin restringe cómo los documentos o scripts cargados desde un origen pueden interactuar con recursos de otro origen. CORS permite habilitar estas interacciones de manera controlada.

```javascript
const cors = require('cors');
app.use(cors());
```

🔹 ¿Cómo implementar caching efectivo en Node.js?

- **En memoria**: Node-cache, Redis.
- **Por capa**: HTTP caching (Cache-Control), reverse proxies (Varnish, NGINX).

🔹 ¿Cómo hacer debugging avanzado en Node.js?

- Usar `node --inspect` y conectar con Chrome DevTools.
- Utilizar breakpoints.
- `console.trace()` para rastrear llamadas.
- Herramientas: `ndb`, `clinic.js`, `debug` package.

🔹 ¿Qué es el Garbage Collector en Node.js y cómo monitorearlo?

El Garbage Collector (GC) libera memoria no utilizada automáticamente.

Monitoreo:

- `--trace-gc` flag.
- Herramientas como Chrome DevTools y `clinic.js`.

🔹 ¿Qué herramientas existen para profiling y performance testing?

- **Profiling**: `clinic.js`, `0x`, Chrome DevTools.
- **Testing**: Artillery, k6, Autocannon.

📚 Recursos adicionales

📖 **Documentación Oficial**:

- [Node.js Docs](https://nodejs.org/en/docs)

🎥 **Videos recomendados**:

- [What the heck is the event loop anyway? - Philip Roberts](https://www.youtube.com/watch?v=8aGhZQkoFbQ)
- [Debugging Memory Leaks in Node.js](https://www.youtube.com/watch?v=8C4ekJj3UjY)

🔗 **Cursos recomendados**:

- [Node.js - The Complete Guide (Udemy)](https://www.udemy.com/course/nodejs-the-complete-guide/)
- [Advanced Node.js (Pluralsight)](https://www.pluralsight.com/courses/nodejs-advanced)